

AUTOMATED ATTENDANCE MANAGEMENT SYSTEM

A Four-Factor Biometric, Geofenced, BLE Proximity Verification and visual dynamic OTP

DEVELOPED FOR:

Final Year Project.

CORE SYSTEM CAPABILITIES:

- GPS Geofencing with Native Android Anti-Mock Protection
- BLE Scan Matching Room Beacons & Visual Time-Sensitive OTPs
- On-device ML Kit Facial Liveness + Server-side Deep Face Recognition

The Attendance Dilemma in Higher Education

The Critical Cost of Manual Attendance

- Traditional manual roll calls take 10 to 15 minutes per lecture, wasting valuable instructional hours across semesters.
- Paper registers are fragile, prone to damage, and offer zero security controls against malicious modifications.
- Compiling reports manually creates immense administrative overhead, causing transcription errors and data delays.
- Physical tracking systems lack an audit trail, making it extremely difficult to verify records during inspections.

Proxy Attendance (Buddy Punching)

Absent students routinely ask classmates to mark them present on sign-in sheets or log responses using shared account links.

Location & Identity Fraud

Students utilize GPS spoofing apps or display a photograph of an absent peer to simple camera scanners to falsify their presence.

QR Code Exploitation

Static QR codes displayed on projectors are instantly photographed and shared via messaging groups, allowing remote sign-ins.

Why Current Automated Systems Fail

Common Technical Flaws & Security Exploits

Static QR Scan Failures

- No Proximity Proof: QR images are sent to external student groups instantly.
- Absent students scan shared QR images from dorms or off-campus locations.
- Zero user validation occurs when the QR scanner acts independently of bios.

GPS Spoofing Flaws

- Android mock location services allow apps to inject custom latitude/longitude.
- GPS drift inside concrete buildings causes false-positives and rejects.
- No verification of whether the student is actually inside the room or outside.

Static Biometric Flaws

- Facial scanners without liveness check can be fooled using prints/screens.
- Device sharing: one student logs into multiple accounts on their phone.
- Heavy server latency when matching raw images without local filtering.

The Four-Factor Verification Shield

01

GPS GEOFENCING

Checks device coordinates against campus boundaries using Android location services, checking and blocking mock providers natively.

02

BLE ROOM PROXIMITY

Scans BLE advertisements from the classroom beacon. Proves physical presence inside the lecture hall via signal attenuation.

03

VISUAL DYNAMIC OTP

A time-sensitive 5-digit hex code generated by the admin. Displayed to students in-class and expires within 10 minutes.

04

LIVENESS & BIOMETRICS

Google ML Kit prompts randomized head turns to verify liveness. Server performs deep 128D facial vector matching.

Project Overview & Ecosystem Mapping

Ecosystem Integration & Workflow Loop

- **Dynamic Classrooms:** Faculty initializes secure, temporary attendance sessions that broadcast dynamic identifiers and visual OTPs.
- **Sensor Fusion Client:** Students utilize a custom Kotlin Android app integrating camera, GPS sensors, and Bluetooth hardware to scan environments.
- **Asynchronous Processing:** The FastAPI backend acts as a gateway, validating physical proximity, verifying identity vectors, and persisting logs.
- **Anti-Proxy Validation:** Verification is failed if any single credential check, location validation, or facial liveness scan is bypassed.
- **Integrated Reporting:** Automatic CSV logs are generated inside classes, tracking cumulative statistics, absences, and registration vectors.

Ecosystem Workflow loop

1. Faculty Starts Session

2. Student Scans
Environment

3. ML Face & Liveness
Engine

4. Save DB & Append
CSV

Built with Modern SaaS Technologies

STUDENT MOBILE CLIENT

- Kotlin: Primary language for Android Native App Development.
- CameraX Jetpack API: Robust and unified camera preview and image capture framework.
- Google ML Kit: Runs on-device face detection and head Euler angles extraction.
- BLE Scanning Engine: Bluetooth Low Energy scanner for beacon filtering.
- Retrofit & OkHttp: Secure network request management with dynamic headers.

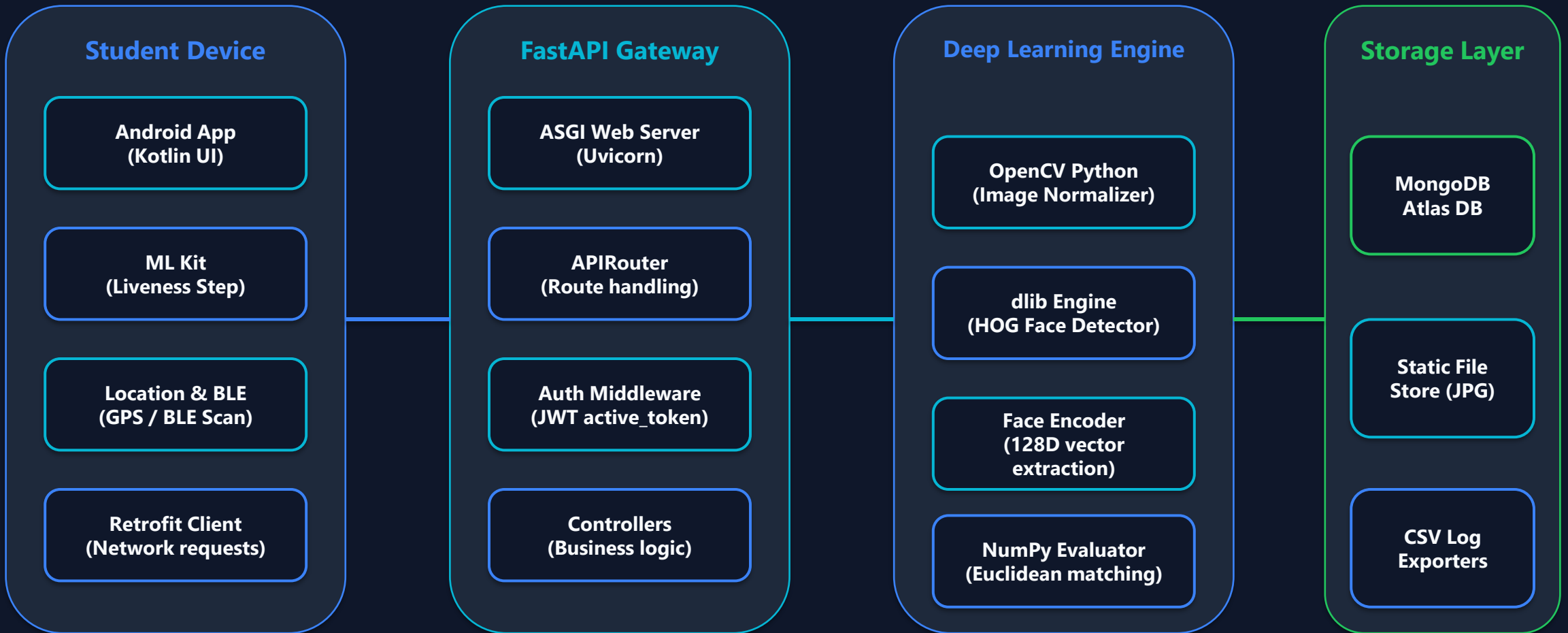
API GATEWAY & BACKEND

- FastAPI: Python framework for high-throughput, async request routing.
- Uvicorn: Lightning-fast ASGI web server implementation.
- JWT Authentication: Cryptographically signed security tokens for users.
- Passlib & BCrypt: Password hashing and security context storage.
- CSV Logger: File-append log engine representing physical sheets.

STORAGE & ML MATCHING

- MongoDB NoSQL: JSON document database storing student and session records.
- dlib / Face Recognition: Computes 128D biometric face vectors on the server.
- OpenCV Python: Resizes and normalizes images for dlib performance.
- NumPy Library: Calculates fast vectorized Euclidean distance for matches.
- PyMongo: Low-latency driver connecting Python directly to database.

End-to-End System Architecture



Modular Application Design

01 ADMIN PORTAL & SYSTEM SETUP

- Class Creation: Define subject names, branches, and schedules.
- Student Registration: Add roll numbers and names to class lists.
- Session Controller: Activate BLE codes and dynamic visual OTPs.
- Report Engine: View present/absent statistics and export CSVs.

02 STUDENT DEVICE UTILITIES

- Ecosystem Select: Log in as a student or connect as admin.
- GPS Verification: Pulls active GPS coordinates from Android.
- BLE Proximity Scan: Searches nearby BLE beacons for classroom validation.
- Visual OTP Verification: Input visual validation codes directly.

03 COMPUTER VISION BIOMETRIC

- ML Kit Analyzer: Local face search, checking yaw and pitch.
- Head-Turn Tracker: Evaluates random directions for anti-spoof.
- Face Cropper: Local scaling & cropping to minimize upload sizes.
- dlib Matcher: Server matches 128D face encodings within 0.5 limit.

04 AUTHENTICATION & MIDDLEWARE

- JWT Management: Secure token generation mapping to admin/student.
- Single-Device Lock: Server compares requests with DB active_token.
- Password Encryption: Bcrypt hashing applied during registration.
- CORS & Security Settings: Restricts API routing to trusted clients.

MongoDB Collection Schemas & Relations

ADMINS

Primary Admin Details

- `_id`: ObjectId
- `name`: String
- `email`: String (Unique)
- `password`: Hash (Bcrypt)

REGISTERED_STUDENTS

Onboarded Accounts

- `_id`: ObjectId
- `name` / `roll` / `email`
- `password`: Hash
- `face_encoding`: Array[128]
- `class_id`: String
- `active_token`: JWT

STUDENTS

Authorized Rolls

- `_id`: ObjectId
- `name` / `roll`
- `class_id`: String
- `class_name`: String
- `attendance_status`: Str

CLASSES

Class & Schedules

- `_id`: ObjectId
- `class_name` / `section`
- `department` / `semester`
- `created_by`: AdminId
- `students`: Array[Objects]
- `expires_at`: DateTime

ATTENDANCE

Attendance Logs

- `_id`: ObjectId
- `student_id`: ObjectId
- `name` / `roll`
- `class_name`: String
- `date`: String (YYYY-MM-DD)
- `status`: Str ('Present')

ATTENDANCE_SESSIONS

Active Live Sessions

- `_id`: ObjectId
- `class_id`: String
- `session_code` / `session_uuid`
- `bluetooth_name`: String
- `classroom_beacon`: String
- `otp_code`: String
- `active`: Boolean

Secure Authentication & Single Device Locking

Student Onboarding (Face Enrollment)

- Pre-Authorization Check: Registration fails if the student's name and roll number are not pre-authorized by an admin in students collection.
- Initial Registration: Users input email, roll number, password, and upload a registration photo.
- Duplicate Face Check: Server extracts a 128D face vector. Scans database registered_students collection for any matching vector (Euclidean distance < 0.5). Blocks duplicate registrations.
- Bcrypt & JWT Setup: Password is hashed using bcrypt. Profile is stored in registered_students collection containing the facial vector.

Login & Session Device Locking (active_token)

- Credential Check: User logs in with email and password via bcrypt comparison.
- Unique JWT Generation: FastAPI creates a unique JWT token embedding user ID and role ('student').
- Session Locking (Database Update): The database updates registered_students with the new JWT as the current 'active_token'.
- Authorization Guard (Middleware): Every request carries the token. The auth dependency compares the request token against active_token in the DB.
- Proxy Defense: If a student shares credentials, a login on a second device updates the active_token. The first device is immediately logged out.

Faculty Admin Session Creation

Steps for starting an attendance session



Student Attendance Marking Journey

Steps to log verification variables and verify attendance



Proximity Proof via BLE Beacons

Bluetooth Low Energy (BLE) Verification Core

- **Scan Window:** Upon starting verification, the student app activates background BLE scanning for exactly 4 seconds.
- **Name Filter:** The app filters all nearby advertisements, checking for names matching the classroom beacon string (e.g. 'CLASS_CSE_A').
- **Signal Strength (RSSI) Check:** Device RSSI values are collected. High attenuation indicates long distance from the beacon.
- **Proximity Proof:** If the expected classroom beacon is detected in the scan map, it confirms the user is inside the physical room.
- **Fail Handling:** If the beacon is missing or RSSI drops below the room threshold, the scan fails. Blocks logins from adjacent corridors or hallways.

BLE Verification Proximity Proof Flow

Teacher Phone / Beacon
CLASS_CSE_A Adv

Student BLE Scanner
4-Second Scan Map

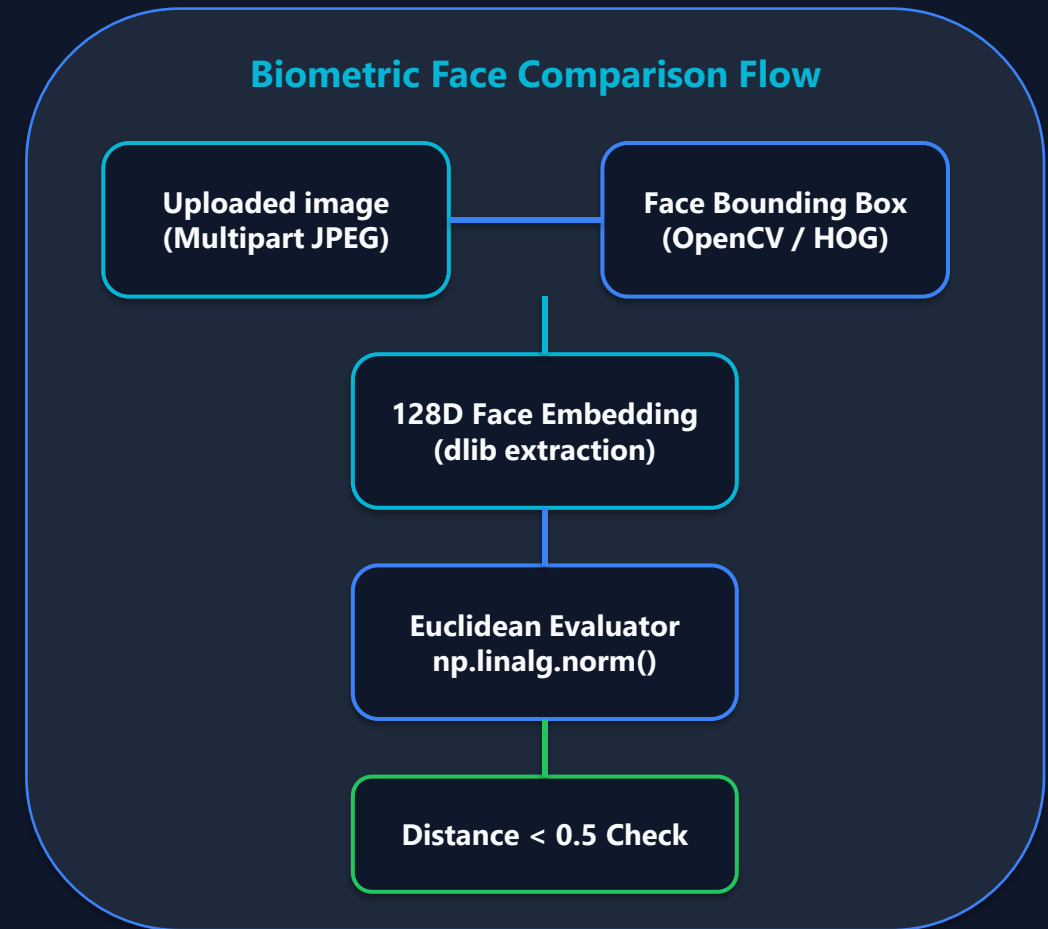
Name Filter &
RSSI Distance Match

Accept: Inside Room
Reject: Mismatch Error

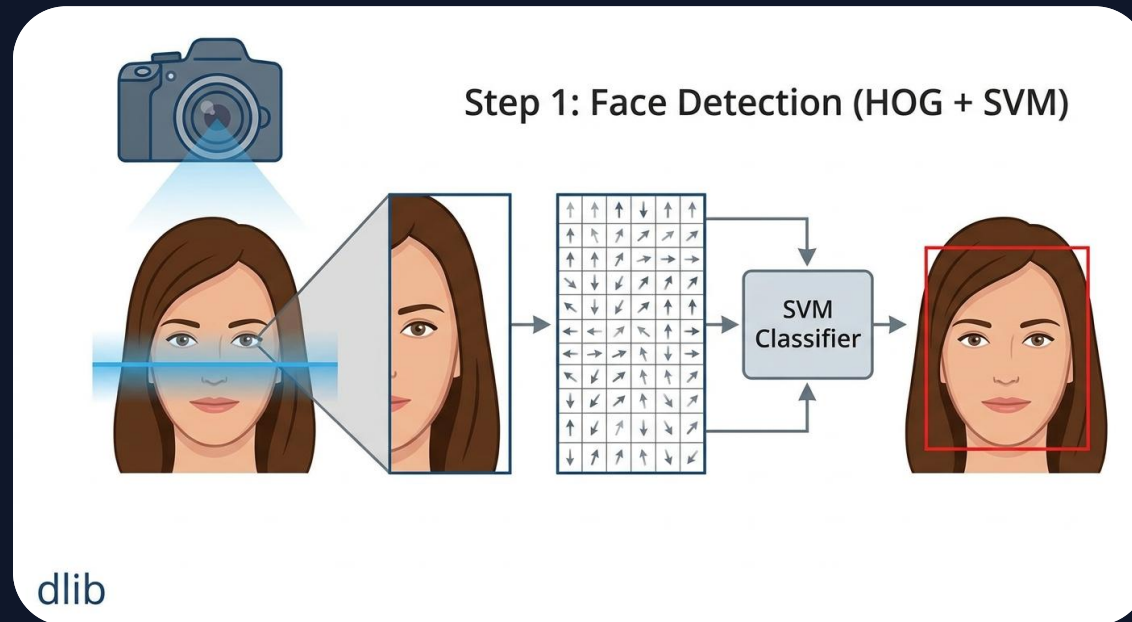
Biometric Face Recognition Flow

Server-Side Facial Embedding Extraction & Distance Scoring

- **Image Upload:** The client sends a cropped front face JPEG via a multipart form to the '/api/attendance/mark' route.
- **Downscaling Preprocessing:** The server resizes the image by 0.5x using OpenCV to speed up localization pipelines.
- **HOG Face Locator:** OpenCV and dlib utilize the Histogram of Oriented Gradients (HOG) algorithm to locate the face bounding box.
- **128-Dimensional Vector:** Dlib's deep learning face recognition model computes a 128D floating-point vector mapping unique facial nodes.
- **Euclidean Distance Check:** Calculates `np.linalg.norm(stored_vector - upload_vector)`. Values < 0.5 indicate a match.



Face Recognition



1. Image Capture

- The camera captures the student's face.

1. HOG Feature Extraction

- Histogram of Oriented Gradients (HOG) analyzes edge directions and facial structures.
- Converts the image into numerical gradient features.

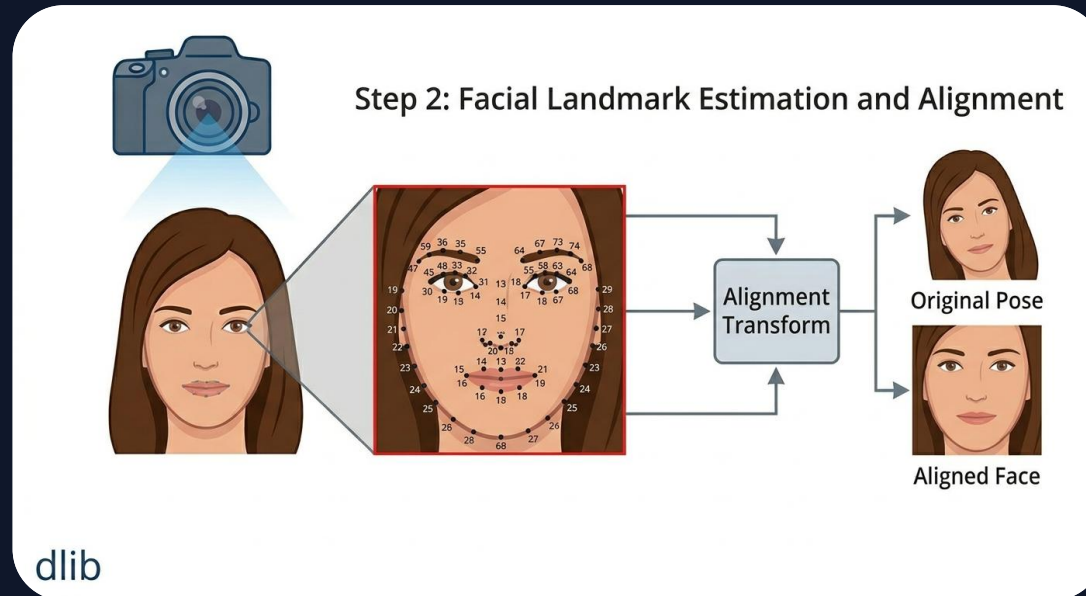
3. SVM Classification

- A Support Vector Machine (SVM) classifier examines the extracted features.
- Determines whether a region contains a face or not.

4. Face Localization

- A bounding box is drawn around the detected face.
- The detected region is passed to the next stage for landmark detection.

Face Recognition



1. Facial Landmark Detection

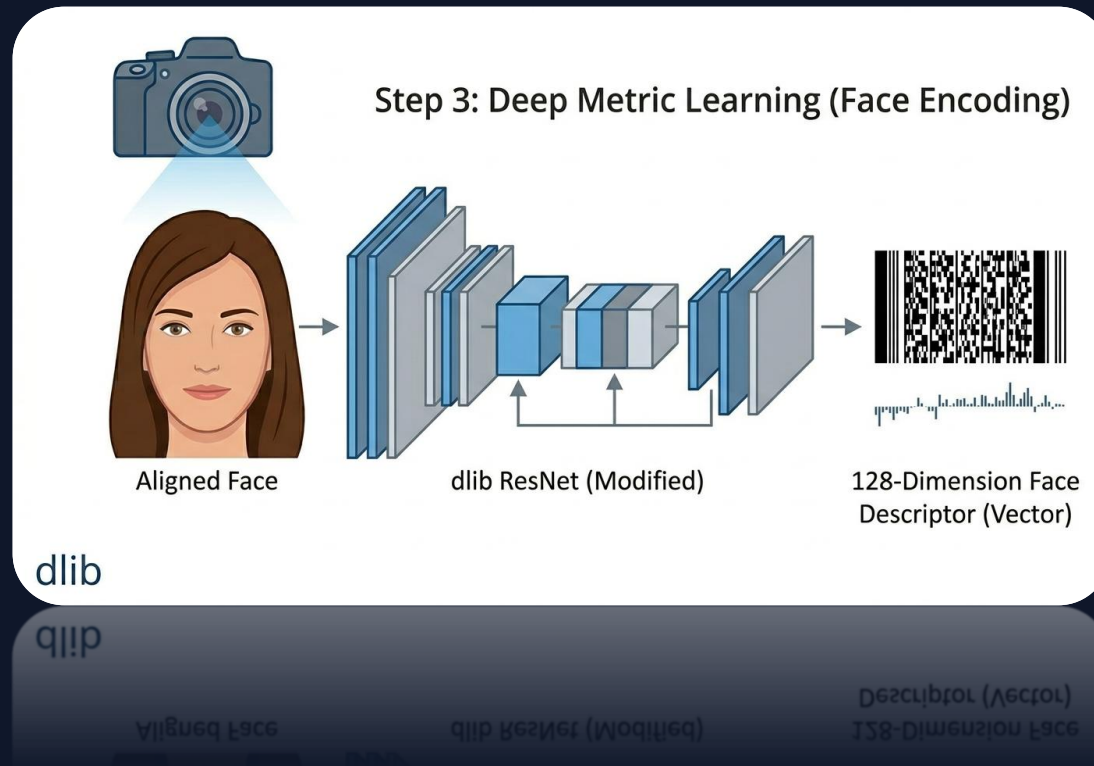
- After detecting a face, dlib identifies **68 facial landmark points**.
- These points represent key facial regions:
 - Eyes
 - Eyebrows
 - Nose
 - Mouth
 - Jawline

2. Landmark Estimation

- Each landmark is assigned precise coordinates on the face.
- The model predicts the geometric structure of the face.
- These landmarks help understand facial orientation and pose.

Jawline	: 17 Points
PointsEyebrows	: 10 Points
PointsEyes	: 12 Points
PointsNose	: 9 Points
PointsMouth	: 20 Points

Face Recognition



1. Aligned Face Input

- Uses the aligned face from the previous stage.
- Ensures consistent facial positioning.

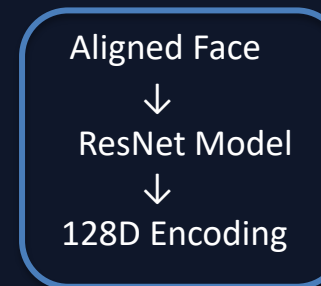
2. Deep Neural Network

- dlib's modified ResNet extracts facial features.
- Learns unique characteristics of each face.

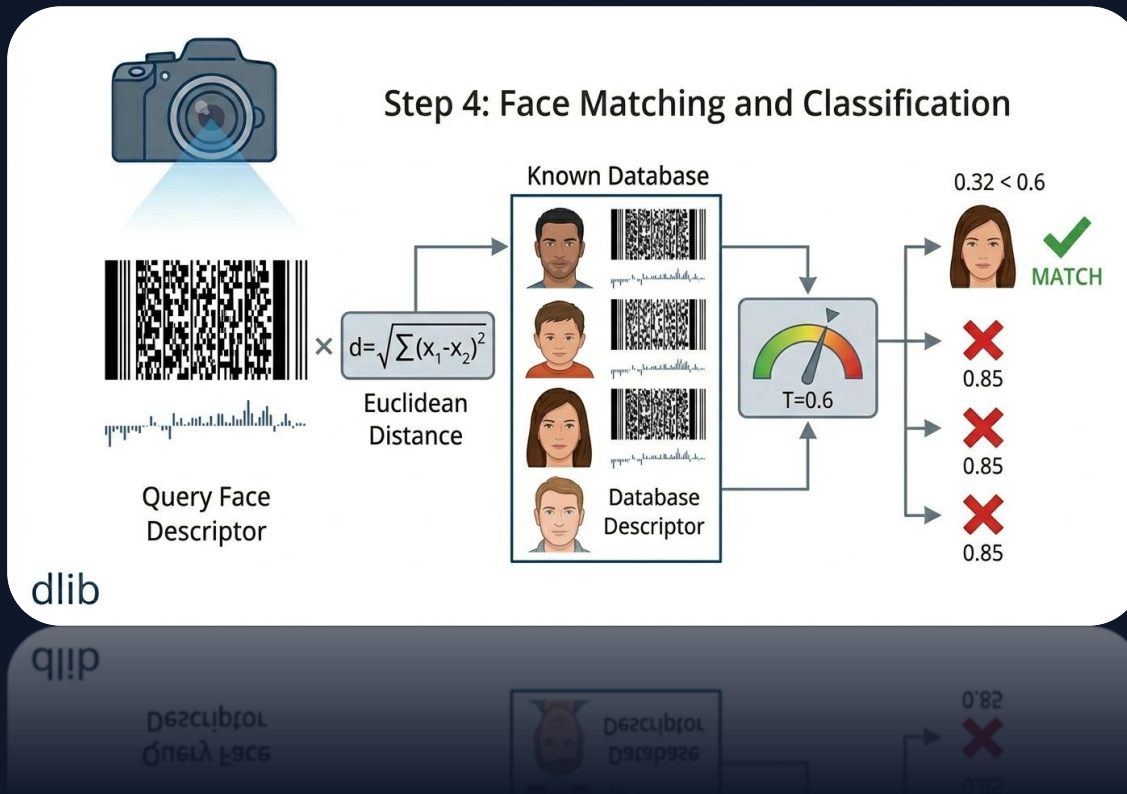
4. Face Encoding

- Converts the face into a **128-D vector**.
- Acts as a mathematical face signature

5. Output



Face Recognition



1. Face Encoding Input

- Uses the generated 128D face encoding.
- Acts as a unique facial signature.

2. Database Comparison

- Compares the query encoding with stored encodings.
- Calculates similarity using Euclidean Distance.

3. Matching Process

- Lower distance indicates higher similarity.
- The closest matching face is selected.

4. Classification

- A bounding box is drawn around the detected face.
- The detected region is passed to the next stage for landmark detection.

Multi-Factor Verification Funnel

Verification Funnel Steps

GPS Geofencing Check (<200 meters of college coords)

Active Session Check & Classroom Beacon Match

Visual dynamic OTP validation (10 min expiry)

Android client head-turn Liveness
Check

Server Face vector matching
(<0.5)

Attendance Marked

Why the funnel structure is used

- **Efficient Cascade:** Sensor validations are run hierarchically to filter out non-compliant requests before running heavy computer vision models.
- **Resource Preservation:** Server-side deep face recognition is only executed if the client has already passed GPS, BLE, OTP, and liveness steps.
- **Cryptographic Integrity:** All factors must compile correctly. Failure at any single point terminates the transaction, preventing false logs.
- **Real-time Updates:** When a student passes the funnel, their database state is changed to 'Present' and written to the class CSV.

Multi-Layered Security Architecture

DATA SECURITY

- Bcrypt Password Hash: User passwords are encrypted with dynamic salt cycles.
- JWT Tokens: Client requests contain cryptographically signed tokens.
- CORS Protocols: Backend restricts API cross-origin routing.
- Secure Storage: Face encodings are stored as numerical arrays, preventing image theft.

CLIENT SENSOR INTEGRITY

- Anti-Mocking: Native Android code flags `location.isFromMockProvider`.
- BLE Proximity: Validates presence using signal power limits (RSSI).
- Visual OTP: Uses in-class displays to block remote code submissions.
- No Local Cache: Verification photos are deleted from memory after upload.

BIOMETRIC PROTECTION

- Face Duplicate Filter: Blocks multiple profiles registering with same face.
- Euler Yaw Checks: Enforces randomized left/right head movements.
- dlib Distance Threshold: Evaluates vector margins at a strict 0.5 limit.
- Aspect Ratio Normalizer: Dynamic crop window limits spoofing borders.

Ecosystem Defenses Against Proxy Attendance

Defeating Attendance Manipulation Hacks

- GPS Mocking Bypass: Defeated! Using Android location API, requests containing mock coordinates are flagged and rejected immediately.
- QR Code Sharing: Defeated! QR scans are replaced by classroom BLE scans and visual OTPs, preventing remote links.
- Photo Spoofing: Defeated! On-device face detection requires active yaw turns (turning left/right) to prove liveness.
- Account Credentials Sharing: Defeated! During user login, the database records the unique active JWT token. Logins on new devices overwrite this token, invalidating old sessions.

Ecosystem Defeat Matrix		
Attack Vector	Proposed Mitigation	Status
GPS spoof / mock locations	Native Android flag check	DEFEATED
Sharing QR screen scans	BLE room scan + visual OTP	DEFEATED
Printed peer photos	Interactive liveness turns	DEFEATED
Sharing credentials	Single active JWT session lock	DEFEATED
Multiple accounts	Registration duplicate face block	DEFEATED

Privacy, Consent, and Data Protection

CONSENT & COLLECTION

- **Explicit Opt-In:** Students must explicitly opt-in to face verification during account onboarding.
- **Zero Image Persistence:** Raw verification photos are processed in memory and immediately discarded.
- **Alternative Verification:** Manual override or OTP verification option is provided for users opting out.

DATA ENCRYPTION

- **Numerical Vectors:** Stores only 128D mathematical vectors; raw images are never kept on the server.
- **Encrypted Identifiers:** Relational database links vectors to hashed profiles, securing student identities.
- **Secure Transmissions:** All client-server request payloads are encrypted via HTTPS protocols.

RETENTION & CONTROL

- **Retention Policy:** Daily attendance logs are archived and purged from active DBs semester-wise.
- **Self-Service Purge:** Students can delete their biometric profiles, instantly removing vectors from NoSQL DB.
- **Access Auditing:** Administrative report access is logged to prevent internal data exposure.

FastAPI REST API Architecture

AUTHENTICATION ROUTER (/api/auth)

- POST /login: Authenticates student; returns JWT; updates active_token.
- POST /register: Saves account details, verifies rolls, saves face vector.
- JWT Verification Middleware: Intercepts requests to validate tokens.

ATTENDANCE CONTROLLER (/api/attendance)

- POST /mark: Multi-factor verification gateway (GPS, BLE, OTP, face upload).
- POST /unmark: Allows admins to reset attendance status to Absent.
- GET /status: Returns daily presence status (Present/Absent).
- GET /history: Returns student's last 7 attendance logs.

SESSION MANAGEMENT (/api/session)

- POST /start/{class_id}: Generates dynamic session identifiers and OTP codes.
- POST /stop/{class_id}: Deactivates attendance window, disabling code inputs.
- GET /active/{class_id}: Client pulls expected beacon and OTP code variables.

CLASS & STUDENT GROUP (/api/class)

- POST /create: Initializes class templates mapping to departments.
- POST /add-students: Uploads roll lists; builds student collections.
- GET /all: Returns active classes mapped to the calling Admin's ID.
- GET /students/{class_id}: Returns detailed classroom lists and attendance stats.

High-Performance System Optimizations

01 VECTORIZED FACE MATCH

Face recognition utilizes NumPy-based Euclidean distance comparisons. Vector matching is run in under 15ms, minimizing CPU bottlenecks.

02 IMAGE DOWNSCALING & PREPROCESSING

Android crops photos to face oval boundaries before uploading. Server downscales images by 0.5x using OpenCV to speed up face localization.

03 DATABASE INDEXES

Indexes are created on roll and email collections in MongoDB, ensuring query times remain under 5ms even during concurrent student access.

04 ASYNCHRONOUS EXPORT

FastAPI writes student records to class CSV spreadsheets in the background, preventing writing operations from blocking API responses.

Infrastructure & System Pre-requisites

Server & Backend Infrastructure

- OS Environment: Ubuntu Server 22.04 LTS recommended, or any Docker-compatible virtual private server (VPS).
- Python ASGI Backend: Python 3.10+ installation with Uvicorn ASGI server for high-concurrency request routing.
- Database Deployment: MongoDB Community Edition for self-hosted setups, or MongoDB Atlas for managed cloud storage.
- Biometric Library Compilation: Pre-installed build tools (CMake, gcc/g++) required to compile C++ dlib dependencies.

Client Devices & In-Class Hardware

- Android Client OS: Android 8.0 Oreo (API Level 26) or higher required to support CameraX API and BLE features.
- Hardware Permissions: User must grant Camera, Fine Location, and Bluetooth Scanning permissions during runtime.
- Classroom BLE Beacons: Bluetooth Low Energy beacons (iBeacon format) deployed and calibrated in each classroom.
- In-Class Displays: Projection screens or monitor setups to show the dynamic visual OTP code during verification windows.

Cost Structure & Financial Feasibility

Estimated Operational Cost Breakdown

Component	Specification	Estimated Cost
Cloud Server VM	2 vCPU, 4GB RAM VPS	\$15 - \$40 / mo
NoSQL Database	MongoDB Atlas (M10 tier)	\$0 - \$30 / mo
Physical BLE Beacons	1 Unit per Classroom	\$15 / room (one-time)
Domain & Security	Custom Domain + SSL	\$10 / year
User App Deployment	Google Play Console fee	\$25 (one-time)

Economic Value & ROI Analysis

- Zero License Fees: Built entirely on open-source dependencies (FastAPI, OpenCV, ML Kit), eliminating recurring user licensing costs.
- Hardware Cost Savings: Eliminates the need for dedicated biometric hardware terminals by leveraging students' existing mobile devices.
- Administrative Efficiency: Pays for itself rapidly by recovering over 15 hours of manual roll-call time per course per semester.
- Scale Flexibility: Free tier database and low-cost server tiers allow initial pilot testing without any upfront software capital expense.

Technical Challenges & Engineering Solutions

01 Android BLE Scan Latency & Mismatch

Challenge: Android BLE scanner takes time to locate and return advertising beacons, causing timeouts.

Resolution: Switched scan settings to Low Latency Mode and created a 4-second scan accumulation map to filter room beacons.

02 Camera Aspect-Ratio Image Warping

Challenge: Raw output from front cameras is stretched or distorted when loading face models, affecting accuracy.

Resolution: Programmed custom cropping logic mapping coordinates directly to CameraX viewfinders.

03 GPS Drift in Concrete Buildings

Challenge: Thick concrete classroom walls block GPS signals, causing drift and false failures.

Resolution: Set GPS checks to a wider 200m range and combined location coordinates with local room BLE scans.

04 Concurrency & Write-blocking

Challenge: Heavy database read/write requests slow down responses during large classroom attendance sessions.

Resolution: Setup indexed fields on student rolls and set FastAPI to write class CSVs asynchronously.

Measurable Benefits & Operational Impact

System Advantages & Value Delivery

- Security Hardening: Eliminates buddy punching, QR sharing, and GPS spoofing completely.
- Administrative Time Savings: Reduces roll call times from 10 minutes to under 5 seconds per student.
- Paperless Tracking: Saves administrative hours by automating classroom statistics and monthly reports.
- Self-contained: Fully open-source dependencies run without expensive third-party APIs.
- Real-time visibility: Dashboard updates instantly when students pass verification.



PROXY ELIMINATION

Significantly reduces common proxy attendance attempts..

< 5s

VERIFICATION SPEED

Fast and smooth student login verification.

0

MANUAL CALCULATION

Automatic real-time Excel/CSV exports.

Next-Gen Attendance Advancements

EDGE FACE RECOGNITION

- Run facial similarity checks locally using TensorFlow Lite on device.
- Saves bandwidth: Uploads only verification hashes instead of full photos.
- Zero server CV overhead: Scales easily to thousands of concurrent users.

MULTI-SPECTRAL LIVENESS

- Incorporate IR/depth sensors on phones supporting advanced biometric API.
- Defeats high-res 3D masks and deepfake screens.
- Continuous verification: Track attendance status throughout the lecture.

AI PREDICTIVE ANALYTICS

- Analyze student attendance trends using machine learning models.
- Predicts semester dropouts and alerts advisors automatically.
- Optimizes classroom schedules by identifying peaks in student absence.

Current System Limitations & Constraints

BIOMETRIC & ENVIRONMENT

- **Lighting Dependency:** Extreme low-light conditions in lecture halls can degrade facial recognition accuracy.
- **Occlusion Sensitivity:** Users must remove heavy facial coverings, sunglasses, or medical masks for biometric scanning.
- **Device Alignment:** Low-resolution front cameras may experience higher vector distance mismatch margins.

HARDWARE & NETWORKS

- **GPS Signal Blockage:** Dense concrete classroom walls can block GPS signals, making BLE scans the critical fallback.
- **Beacon Range Calibration:** Requiring precise transmission power calibration to prevent corridor/adjacent room bleeding.
- **Platform Lock-in:** Native client software is currently restricted to Android devices, lacking iOS support.

OPERATIONAL SYSTEM

- **Onboarding Overhead:** Requires a one-time onboarding enrollment step for face vector database initialization.
- **Proxy Token Sharing:** While JWT session locks block multi-device logins, they do not prevent students from sharing OTPs offsite if BLE is bypassed.
- **Server Latency:** Simultaneous uploads of image binaries at the end of a session can cause brief queue bottlenecks.

Client Mobile Interface Walkthrough

Key User Interface Screens & Interactive Elements

01

SPLASH & ROLE SELECT

Interactive role select with custom canvas mesh backgrounds.

02

LOGIN & REGISTER

Includes email validation, passwords, and face enrollment.

03

STUDENT DASHBOARD

Shows weekly graphs, active sessions, and attendance statuses.

04

CODE INPUT DIALOG

A Material Design popup checking dynamic visual OTP codes.

05

BLE SCANNER VIEW

Real-time background scanner filtering room beacons.

06

LIVENESS OVERLAY

Draws face framing ovals and arrow directions for head turns.

07

SUCCESS SCREEN

Lottie animation playing on verified attendance marks.

08

REPORTS & EXPORT

Class list overview displaying real-time CSV statistics.

Securing Modern Classrooms

Project Conclusion & Accomplishments

- Successfully designed, implemented, and verified a secure, multi-factor automated attendance management system.
- Integrated mobile sensors, local ML analytics, and server-side biometrics to create a robust security shield.
- Eliminated attendance vulnerabilities (GPS mocking, QR sharing, buddy punching, static photos).
- Greatly reduced administrative load for teachers by automating statistics and spreadsheet logs.
- Ecosystem ready for deployment across academic and enterprise training environments.

KEY DELIVERABLE METRICS

- Secure: Low Euclidean matching limits.
- Fast: Complete verification takes under 5s.
- Automated: Logs compiled in real-time CSV formats.
- No Spoofing: Robust head-turning liveness detection.
- On-device Processing: Client-side facial analysis.

THANK YOU!

Questions & Feedback

AUTOMATED ATTENDANCE MANAGEMENT SYSTEM

- Ecosystem Source: github.com/Slothyi/Automated-attendance-management-system
- Technical Architecture: Python FastAPI Gateway | Kotlin Android App
- Biometrics Engine: Google ML Kit Face Analysis & dlib 128D Matching